

POLY

POLYMORPHISM AND HIGHER-ORDER FUNCTIONS

From `LF Require Export Lists`.

Polymorphism

In this chapter we continue our development of basic concepts of functional programming. The critical new ideas are *polymorphism* (abstracting functions over the types of the data they manipulate) and *higher-order functions* (treating functions as data). We begin with polymorphism.

Polymorphic Lists

In the last chapter, we worked with lists containing just numbers. Obviously, interesting programs also need to be able to manipulate lists with elements from other types -- lists of booleans, lists of lists, etc. We *could* just define a new inductive datatype for each of these, for example...

```
Inductive boollist : Type :=  
  | bool_nil  
  | bool_cons (b : bool) (l : boollist).
```

... but this would quickly become tedious: not only would we have to make up different constructor names for each datatype, but -- even worse -- we would also need to define new versions of all the list manipulating functions (`length`, `app`, `rev`, etc.) and all their properties (`rev_length`, `app_assoc`, etc.) for each new definition.

To avoid all this repetition, Rocq supports *polymorphic* inductive type definitions. For example, here is a *polymorphic list* datatype.

```
Inductive list (X:Type) : Type :=  
  | nil  
  | cons (x : X) (l : list X).
```

This is exactly like the definition of `natlist` from the previous chapter, except that the `nat` argument to the `cons` constructor has been replaced by an arbitrary type `X`, a binding for `X` has been added to the function header on the first line, and the occurrences of `natlist` in the types of the constructors have been replaced by `list X`. We can now write `list nat` instead of `natlist`.

What sort of thing is `list` itself? A good way to think about it is that the definition of `list` is a *function* from `Types` to `Inductive definitions`; or, to put it more concisely, `list` is a function from `Types` to `Types`. For any particular type `X`, the type `list X` is the Inductively defined set of lists whose elements are of type `X`.

```
Check list : Type → Type.
```

The `X` in the definition of `list` automatically becomes a parameter to the constructors `nil` and `cons` -- that is, `nil` and `cons` are now polymorphic constructors; when we use them, we must provide, as a first argument, the type of the list they are building. For example, `nil nat` is the empty list of type `nat`.

```
Check (nil nat) : list nat.
```

Similarly, `cons nat` adds an element of type `nat` to a list of type `list nat`. Here is an example of forming a list containing just the natural number 3.

```
Check (cons nat 3 (nil nat)) : list nat.
```

What might the type of `nil` be? We can read off the type `list X` from the definition, but this omits the binding for `X` which is the parameter to `list`. `Type → list X` does not explain the meaning of `X`. `(X : Type) → list X` comes closer. Rocq's notation for this situation is $\forall X : \text{Type}, \text{list } X$.

```
Check nil : ∀ X : Type, list X.
```

Similarly, the type of `cons` from the definition looks like `X → list X → list X`, but using this convention to explain the meaning of `X` results in the type $\forall X, X \rightarrow \text{list } X \rightarrow \text{list } X$.

```
Check cons : ∀ X : Type, X → list X → list X.
```

(A side note on notations: In `.v` files, the "forall" quantifier is spelled out in letters. In the corresponding HTML files (and in the way some IDEs show `.v` files, depending on the settings of their display controls), $\forall$ is usually typeset as the standard mathematical "upside down A," though you'll still see the spelled-out "forall" in a few places. This is just a quirk of typesetting -- there is no difference in meaning.)

Having to supply a type argument for every single use of a list constructor would be rather burdensome; we will soon see ways of reducing this annotation burden.

```
Check (cons nat 2 (cons nat 1 (nil nat)))
      : list nat.
```

We can now go back and make polymorphic versions of all the list-processing functions that we wrote before. Here is `repeat`, for example:

```
Fixpoint repeat (X : Type) (x : X) (count : nat) : list X :=
  match count with
  | 0 ⇒ nil X
  | S count' ⇒ cons X x (repeat X x count')
  end.
```

As with `nil` and `cons`, we can use `repeat` by applying it first to a type and then to an element of this type (and a number):

```
Example test_repeat1 :
  repeat nat 4 2 = cons nat 4 (cons nat 4 (nil nat)).
+
```

To use `repeat` to build other kinds of lists, we simply instantiate it with an appropriate type parameter:

```
Example test_repeat2 :
  repeat bool false 1 = cons bool false (nil bool).
+
```

Exercise: 2 stars, standard, optional (mumble grumble)

Consider the following two inductively defined types.

```
Module MumbleGrumble.

Inductive mumble : Type :=
| a
| b (x : mumble) (y : nat)
| c.

Inductive grumble (X:Type) : Type :=
| d (m : mumble)
| e (x : X).
```

Which of the following are well-typed elements of `grumble X` for some type `X`?
(Add YES or NO to each line.)

- `d (b a 5)`
- `d mumble (b a 5)`
- `d bool (b a 5)`
- `e bool true`
- `e mumble (b c 0)`
- `e bool (b c 0)`
- `c`

```
(* FILL IN HERE *)
End MumbleGrumble.
```

□

Type Annotation Inference

Let's write the definition of `repeat` again, but this time we won't specify the types of any of the arguments. Will Rocq still accept it?

```
Fixpoint repeat' X x count : list X :=
  match count with
  | 0 => nil X
  | S count' => cons X x (repeat' X x count')
  end.
```

Indeed it will. Let's see what type Rocq has assigned to `repeat'` ...

```
Check repeat'
: ∀ X : Type, X → nat → list X.
Check repeat
: ∀ X : Type, X → nat → list X.
```

It has exactly the same type as `repeat`. Rocq was able to use *type inference* to deduce what the types of `X`, `x`, and `count` must be, based on how they are used. For example, since `X` is used as an argument to `cons`, it must be a `Type`, since `cons` expects a `Type` as its first argument; matching `count` with `0` and `S` means it must be a `nat`; and so on.

This powerful facility means we don't always have to write explicit type annotations everywhere, although explicit type annotations can still be quite useful as documentation and sanity checks, so we will continue to use them much of the time.

Type Argument Synthesis

To use a polymorphic function, we need to pass it one or more types in addition to its other arguments. For example, the recursive call in the body of the `repeat` function above must pass along the type `X`. But since the second argument to `repeat` is an element of `X`, it seems entirely obvious that the first argument can only be `X` -- why should we have to write it explicitly?

Fortunately, Rocq permits us to avoid this kind of redundancy. In place of any type argument we can write a "hole" `_`, which can be read as "Please try to figure out for yourself what belongs here." More precisely, when Rocq encounters a `_`, it will attempt to *unify* all locally available information -- the type of the function being applied, the types of the other arguments, and the type expected by the context in which the application appears -- to determine what concrete type should replace the `_`.

This may sound similar to type annotation inference -- and, indeed, the two procedures rely on the same underlying mechanisms. Instead of simply omitting the types of some arguments to a function, like

```
repeat' X x count : list X :=
```

we can also replace the types with holes

```
repeat' (X : _) (x : _) (count : _) : list X :=
```

to tell Rocq to attempt to infer the missing information.

Using holes, the `repeat` function can be written like this:

```
Fixpoint repeat'' X x count : list X :=
  match count with
  | 0 => nil
  | S count' => cons _ x (repeat'' _ x count')
end.
```

In this instance, we don't save much by writing `_` instead of `X`. But in many cases the difference in both keystrokes and readability is nontrivial. For example, suppose we want to write down a list containing the numbers `1`, `2`, and `3`. Instead of this...

```
Definition list123 :=
  cons nat 1 (cons nat 2 (cons nat 3 (nil nat))).
```

...we can use holes to write this:

```
Definition list123' :=
  cons _ 1 (cons _ 2 (cons _ 3 (nil _))).
```

Implicit Arguments

In fact, we can go further and even avoid writing `_`'s in most cases by telling Rocq *always* to infer the type argument(s) of a given function.

The `Arguments` directive specifies the name of the function (or constructor) and then lists the (leading) argument names to be treated as implicit, each surrounded by curly braces.

```
Arguments nil {X}.
Arguments cons {X}.
Arguments repeat {X}.
```

Now we don't have to supply any type arguments at all in the example:

```
Definition list123'' := cons 1 (cons 2 (cons 3 nil)).
```

Alternatively, we can declare an argument to be implicit when defining the function itself, by surrounding it in curly braces instead of parens. For example:

```
Fixpoint repeat'' {X : Type} (x : X) (count : nat) : list X :=
  match count with
  | 0 => nil
  | S count' => cons x (repeat'' x count')
  end.
```

(Note that we didn't even have to provide a type argument to the recursive call to `repeat''`. Indeed, it would be invalid to provide one, because Rocq is not expecting it.)

We will use the latter style whenever possible, but we will continue to use explicit **Argument** declarations for **Inductive** constructors. The reason for this is that marking the parameter of an inductive type as implicit causes it to become implicit for the type itself, not just for its constructors. For instance, consider the following alternative definition of the `list` type:

```
Inductive list' {X:Type} : Type :=
  | nil'
  | cons' (x : X) (l : list').
```

Because `X` is declared as implicit for the *entire* inductive definition including `list'` itself, we now have to write just `list'` whether we are talking about lists of numbers or booleans or anything else, rather than `list' nat` or `list' bool` or whatever; this is a step too far.

Let's finish by re-implementing a few other standard list functions on our new polymorphic lists...

```
Fixpoint app {X : Type} (l1 l2 : list X) : list X :=
  match l1 with
  | nil => l2
  | cons h t => cons h (app t l2)
  end.
```

```
Fixpoint rev {X:Type} (l:list X) : list X :=
  match l with
  | nil => nil
  | cons h t => app (rev t) (cons h nil)
  end.
```

```
Fixpoint length {X : Type} (l : list X) : nat :=
  match l with
  | nil => 0
  | cons _ l' => S (length l')
  end.
```

```
Example test_rev1 :
  rev (cons 1 (cons 2 nil)) = (cons 2 (cons 1 nil)).
+
```

```
Example test_rev2:
  rev (cons true nil) = cons true nil.
+
```

```
Example test_length1: length (cons 1 (cons 2 (cons 3 nil))) = 3.
+
```

Supplying Type Arguments Explicitly

One small problem with declaring arguments to be implicit is that, once in a while, Rocq does not have enough local information to determine a type argument; in such cases, we need to tell Rocq that we want to give the argument explicitly just this time. For example, suppose we write this:

```
Fail Definition mynil := nil.
```

(The `Fail` qualifier that appears before `Definition` can be used with *any* command, and is used to ensure that that command indeed fails when executed. If the command does fail, Rocq prints the corresponding error message, but continues processing the rest of the file.)

Here, Rocq gives us an error because it doesn't know what type argument to supply to `nil`. We can help it by providing an explicit type declaration (so that Rocq has more information available when it gets to the "application" of `nil`):

```
Definition mynil : list nat := nil.
```

Alternatively, we can disable the implicit argument to `nil` by prefixing the function name with `@`. This allows us to apply `@nil` *explicitly* to an appropriate type.

```
Check @nil : ∀ X : Type, list X.
```

```
Definition mynil' := @nil nat.
```

(Note that we cannot just write `nil nat` here, without the `@`: the `Implicit Arguments` declaration above means that `nil nat` is now interpreted by Rocq as `@nil ?X nat`, where `?X` is some unknown type that should be filled in by implicit argument synthesis.)

```
Fail Check (nil nat).
```

Using argument synthesis and implicit arguments, we can define convenient notation for lists, as before. Since we have made the constructor type arguments implicit, Rocq will know to automatically infer these when we use the notations.

```
Notation "x :: y" := (cons x y)
                      (at level 60, right associativity).
Notation "[ ]" := nil.
Notation "[ x ; .. ; y ]" := (cons x .. (cons y []) ..).
Notation "x ++ y" := (app x y)
                      (at level 60, right associativity).
```

Now lists can be written just the way we'd hope:

```
Definition list123''' := [1; 2; 3].
```

Exercises

Exercise: 2 stars, standard (poly exercises)

Here are a few simple exercises, just like ones in the `Lists` chapter, for practice with polymorphism. Complete the proofs below.

```
Theorem app_nil_r : ∀ (X : Type), ∀ l : list X,
  l ++ [] = l.
Proof.
  (* FILL IN HERE *) Admitted.
```

```
Theorem app_assoc : ∀ A (l m n : list A),
  l ++ m ++ n = (l ++ m) ++ n.
```

```

Proof.
  (* FILL IN HERE *) Admitted.

Lemma app_length : ∀ (X : Type) (l1 l2 : list X),
  length (l1 ++ l2) = length l1 + length l2.
Proof.
  (* FILL IN HERE *) Admitted.
□

```

Exercise: 2 stars, standard (more poly exercises)

Here are some slightly more interesting ones...

```

Theorem rev_app_distr: ∀ X (l1 l2 : list X),
  rev (l1 ++ l2) = rev l2 ++ rev l1.
Proof.
  (* FILL IN HERE *) Admitted.

Theorem rev_involutive : ∀ X : Type, ∀ l : list X,
  rev (rev l) = l.
Proof.
  (* FILL IN HERE *) Admitted.
□

```

Polymorphic Pairs

Following the same pattern, the definition for pairs of numbers that we gave in the last chapter can be generalized to *polymorphic pairs*, often called *products*:

```

Inductive prod (X Y : Type) : Type :=
| pair (x : X) (y : Y).

Arguments pair {X} {Y}.

```

As with lists, we make the type arguments implicit and define the familiar concrete notation.

```

Notation "( x , y )" := (pair x y).

```

We can also use the `Notation` mechanism to define the standard notation for *product types* (i.e., the types of pairs):

```

Notation "X * Y" := (prod X Y) : type_scope.

```

(The annotation `: type_scope` tells Rocq that this abbreviation should only be used when parsing types, not when parsing expressions. This avoids a clash with the multiplication symbol.)

It is easy at first to get (x, y) and $X \times Y$ confused. Remember that (x, y) is a *value* built from two other values, while $X \times Y$ is a *type* built from two other types. If x has type X and y has type Y , then (x, y) has type $X \times Y$.

The first and second projection functions now look pretty much as they would in any functional programming language.

```

Definition fst {X Y : Type} (p : X × Y) : X :=
  match p with
  | (x, y) => x
  end.

Definition snd {X Y : Type} (p : X × Y) : Y :=
  match p with
  | (x, y) => y
  end.

```

The following function takes two lists and combines them into a list of pairs. In other functional languages, it is often called `zip`; we call it `combine` for consistency with Rocq's standard library.

```
Fixpoint combine {X Y : Type} (lx : list X) (ly : list Y)
  : list (X×Y) :=
  match lx, ly with
  | [], _ ⇒ []
  | _, [] ⇒ []
  | x :: tx, y :: ty ⇒ (x, y) :: (combine tx ty)
  end.
```

Exercise: 1 star, standard, optional (combine checks)

Try answering the following questions on paper and checking your answers in Rocq:

- What is the type of `combine` (i.e., what does `Check @combine` print?)
- What does `Compute (combine [1;2] [false;false;true;true]).` print?

□

Exercise: 2 stars, standard, especially useful (split)

The function `split` is the right inverse of `combine`: it takes a list of pairs and returns a pair of lists. In many functional languages, it is called `unzip`.

Fill in the definition of `split` below. Make sure it passes the given unit test.

```
Fixpoint split {X Y : Type} (l : list (X×Y)) : (list X) × (list Y)
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.
```

```
Example test_split:
  split [(1,false);(2,false)] = ([1;2],[false;false]).
Proof.
(* FILL IN HERE *) Admitted.
```

□

Polymorphic Options

Our last polymorphic type for now is *polymorphic options*, which generalize `natoption` from the previous chapter. (We put the definition inside a module because the standard library already defines `option` and it's this one that we want to use below.)

```
Module OptionPlayground.

Inductive option (X:Type) : Type :=
  | Some (x : X)
  | None.

Arguments Some {X}.
Arguments None {X}.

End OptionPlayground.
```

We can now rewrite the `nth_error` function so that it works with any type of lists.

```
Fixpoint nth_error {X : Type} (l : list X) (n : nat)
  : option X :=
  match l with
  | nil ⇒ None
```



```

| a :: l' => match n with
| 0 => Some a
| S n' => nth_error l' n'
end

end.

Example test_nth_error1 : nth_error [4;5;6;7] 0 = Some 4.
+
Example test_nth_error2 : nth_error [[1];[2]] 1 = Some [2].
+
Example test_nth_error3 : nth_error [true] 2 = None.
+

```

Exercise: 1 star, standard, optional (hd_error_poly)

Complete the definition of a polymorphic version of the `hd_error` function from the last chapter. Be sure that it passes the unit tests below.

```

Definition hd_error {X : Type} (l : list X) : option X
(* REPLACE THIS LINE WITH ":= _your_definition_" *). Admitted.

```

Once again, to force the implicit arguments to be explicit, we can use `@` before the name of the function.

```

Check @hd_error : ∀ X : Type, list X → option X.

Example test_hd_error1 : hd_error [1;2] = Some 1.
(* FILL IN HERE *) Admitted.
Example test_hd_error2 : hd_error [[1];[2]] = Some [1].
(* FILL IN HERE *) Admitted.

```

□

Functions as Data

Like most modern programming languages -- especially other "functional" languages, including OCaml, Haskell, Racket, Scala, Clojure, etc. -- Rocq treats functions as first-class citizens, allowing them to be passed as arguments to other functions, returned as results, stored in data structures, etc.

Higher-Order Functions

Functions that manipulate other functions are often called *higher-order* functions. Here's a simple one:

```

Definition doit3times {X : Type} (f : X → X) (n : X) : X :=
  f (f (f n)).

```

The argument `f` here is itself a function (from `X` to `X`); the body of `doit3times` applies `f` three times to some value `n`.

```

Check @doit3times : ∀ X : Type, (X → X) → X → X.

Example test_doit3times: doit3times minustwo 9 = 3.
+

Example test_doit3times': doit3times negb true = false.
+

```

Filter

Here is a more useful higher-order function, taking a list of X s and a *predicate* on X (a function from X to `bool`) and "filtering" the list to yield a new list containing just those elements for which the predicate returns `true`.

```
Fixpoint filter {X : Type} (test : X→bool) (l : list X) : list X :=
  match l with
  | [] => []
  | h :: t =>
    if test h then h :: (filter test t)
    else filter test t
  end.
```

For example, if we apply `filter` to the predicate `even` and a list of numbers `l`, it returns a list containing just the even members of `l`.

```
Example test_filter1: filter even [1;2;3;4] = [2;4].
+
```

```
Definition length_is_1 {X : Type} (l : list X) : bool :=
  (length l) =? 1.
```

```
Example test_filter2:
  filter length_is_1
    [ [1; 2]; [3]; [4]; [5;6;7]; []; [8] ]
  = [ [3]; [4]; [8] ].
+
```

We can use `filter` to give a concise version of the `countoddmembers` function from the [Lists](#) chapter.

```
Definition countoddmembers' (l : list nat) : nat :=
  length (filter odd l).
```

```
Example test_countoddmembers'1: countoddmembers' [1;0;3;1;4;5] = 4.
```

```
Example test_countoddmembers'2: countoddmembers' [0;2;4] = 0.
```

```
Example test_countoddmembers'3: countoddmembers' nil = 0.
```

Anonymous Functions

It is arguably a little sad, in the example just above, to be forced to define the function `length_is_1` and give it a name just to be able to pass it as an argument to `filter`, since we will probably never use it again. Indeed, when using higher-order functions, we *often* want to pass as arguments "one-off" functions that we will never use again; having to give each of these functions a name would be tedious.

Fortunately, there is a better way. We can construct a function "on the fly" without declaring it at the top level or giving it a name.

```
Example test_anon_fun':
  doit3times (fun n => n × n) 2 = 256.
```

The expression `(fun n => n × n)` can be read as "the function that, given a number n , yields $n \times n$."

Here is the `filter` example, rewritten to use an anonymous function.

```
Example test_filter2':
  filter (fun l => (length l) =? 1)
```

```

      [ [1; 2]; [3]; [4]; [5;6;7]; []; [8] ]
    = [ [3]; [4]; [8] ].
+

```

Exercise: 2 stars, standard (filter_even_gt7)

Use `filter` (instead of `Fixpoint`) to write a Rocq function `filter_even_gt7` that takes a list of natural numbers as input and returns a list of just those that are even and greater than 7.

```

Definition filter_even_gt7 (l : list nat) : list nat
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.

```

```

Example test_filter_even_gt7_1 :
  filter_even_gt7 [1;2;6;9;10;3;12;8] = [10;12;8].
(* FILL IN HERE *) Admitted.

```

```

Example test_filter_even_gt7_2 :
  filter_even_gt7 [5;2;6;19;129] = [].
(* FILL IN HERE *) Admitted.

```

□

Exercise: 3 stars, standard (partition)

Use `filter` to write a Rocq function `partition` that, given a set X , a predicate of type $X \rightarrow \text{bool}$ and a `list X`, should return a pair of lists. The first member of the pair is the sublist of the original list containing the elements that satisfy the test, and the second is the sublist containing those that fail the test. The order of elements in the two sublists should be the same as their order in the original list.

```

Definition partition {X : Type}
  (test : X → bool)
  (l : list X)
  : list X × list X
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.

```

```

Example test_partition1: partition odd [1;2;3;4;5] = ([1;3;5],
[2;4]).
(* FILL IN HERE *) Admitted.
Example test_partition2: partition (fun x ⇒ false) [5;9;0] = ([],
[5;9;0]).
(* FILL IN HERE *) Admitted.

```

□

Map

Another handy higher-order function is called `map`.

```

Fixpoint map {X Y : Type} (f : X→Y) (l : list X) : list Y :=
  match l with
  | [] ⇒ []
  | h :: t ⇒ (f h) :: (map f t)
  end.

```

It takes a function f and a list $l = [n_1, n_2, n_3, \dots]$ and returns the list $[f\ n_1, f\ n_2, f\ n_3, \dots]$, where f has been applied to each element of l in turn. For example:

```

Example test_map1: map (fun x ⇒ plus 3 x) [2;0;2] = [5;3;5].
+

```

The element types of the input and output lists need not be the same, since `map` takes *two* type arguments, `X` and `Y`; it can thus be applied to a list of numbers and a function from numbers to booleans to yield a list of booleans:

```
Example test_map2:
  map odd [2;1;2;5] = [false;true;false;true].
+
```

It can even be applied to a list of numbers and a function from numbers to *lists* of booleans to yield a *list of lists* of booleans:

```
Example test_map3:
  map (fun n => [even n;odd n]) [2;1;2;5]
  = [[true;false];[false;true];[true;false];[false;true]].
+
```

Exercises

Exercise: 3 stars, standard (map_rev)

Show that `map` and `rev` commute. (Hint: You may need to define an auxiliary lemma.)

```
Theorem map_rev : ∀ (X Y : Type) (f : X → Y) (l : list X),
  map f (rev l) = rev (map f l).
Proof.
  (* FILL IN HERE *) Admitted.
□
```

Exercise: 2 stars, standard, especially useful (flat_map)

The function `map` maps a `list X` to a `list Y` using a function of type `X → Y`. We can define a similar function, `flat_map`, which maps a `list X` to a `list Y` using a function `f` of type `X → list Y`. Your definition should work by 'flattening' the results of `f`, like so:

```
flat_map (fun n => [n;n+1;n+2]) [1;5;10]
= [1; 2; 3; 5; 6; 7; 10; 11; 12].

Fixpoint flat_map {X Y: Type} (f: X → list Y) (l: list X)
  : list Y
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.

Example test_flat_map1:
  flat_map (fun n => [n;n;n]) [1;5;4]
  = [1; 1; 1; 5; 5; 5; 4; 4; 4].
  (* FILL IN HERE *) Admitted.
□
```

Lists are not the only inductive type for which `map` makes sense. Here is a `map` for the `option` type:

```
Definition option_map {X Y : Type} (f : X → Y) (xo : option X)
  : option Y :=
  match xo with
  | None => None
  | Some x => Some (f x)
  end.
```

Exercise: 2 stars, standard, optional (implicit_args)

The definitions and uses of `filter` and `map` use implicit arguments in many places. Replace the curly braces around the implicit arguments with parentheses, and then fill in explicit type parameters where necessary and use Rocq to check that you've done so correctly. (This exercise is not to be turned in; it is probably easiest to do it on a *copy* of this file that you can throw away afterwards.) □

Fold

An even more powerful higher-order function is called `fold`. This function is the inspiration for the "reduce" operation that lies at the heart of Google's map/reduce distributed programming framework.

```
Fixpoint fold {X Y: Type} (f : X → Y → Y) (l : list X) (b : Y)
  : Y :=
  match l with
  | nil ⇒ b
  | h :: t ⇒ f h (fold f t b)
  end.
```

Intuitively, the behavior of the `fold` operation is to insert a given binary operator `f` between every pair of elements in a given list. For example, `fold plus [1;2;3;4]` intuitively means `1+2+3+4`. To make this precise, we also need a "starting element" that serves as the initial second input to `f`. So, for example,

```
fold plus [1;2;3;4] 0
yields
  1 + (2 + (3 + (4 + 0))).
]] *)
```

```
Example fold_example1 :
  fold andb [true;true;false;true] true = false.
+
```

```
Example fold_example2 :
  fold mult [1;2;3;4] 1 = 24.
+
```

```
Example fold_example3 :
  fold app [[1];[];[2;3];[4]] [] = [1;2;3;4].
+
```

```
Example foldexample4 :
  fold (fun l n ⇒ length l + n) [[1];[];[2;3;2];[4]] 0 = 5.
Proof. reflexivity. Qed.
```

Exercise: 1 star, standard, optional (fold types different)

Observe that the type of `fold` is parameterized by *two* type variables, `X` and `Y`, and the parameter `f` is a binary operator that takes an `X` and a `Y` and returns a `Y`. Example `foldexample4` above shows one instance where it is useful for `X` and `Y` to be different. Can you think of any others?

```
(* FILL IN HERE *)
```

□

Functions That Construct Functions

Most of the higher-order functions we have talked about so far take functions as arguments. Let's look at some examples that involve *returning* functions as the results of other functions. To begin, here is a function that takes a value `x` (drawn from some type `X`) and returns a function from `nat` to `X` that yields `x` whenever it is called, ignoring its `nat` argument.

```
Definition constfun {X : Type} (x : X) : nat → X :=
  fun (k:nat) => x.
```

```
Definition ftrue := constfun true.
```

```
Example constfun_example1 : ftrue 0 = true.
```

+

```
Example constfun_example2 : (constfun 5) 99 = 5.
```

+

In fact, the multiple-argument functions we have already seen are also examples of passing functions as data. To see why, recall the type of `plus`.

```
Check plus : nat → nat → nat.
```

```
Definition plus3 := plus 3.
```

```
Check plus3 : nat → nat.
```

```
Example test_plus3 : plus3 4 = 7.
```

+

```
Example test_plus3' : doit3times plus3 0 = 9.
```

+

```
Example test_plus3'' : doit3times (plus 3) 0 = 9.
```

+

Similarly, we can write:

```
Definition fold_plus :=
  fold plus.
```

```
Check fold_plus : list nat → nat → nat.
```

What's happening here is called **partial application**. In Rocq, the type constructor `→` is right-associative, meaning a function type like `A → B → C` is parsed like `A → (B → C)`, or "a function from `A` to a function from `B` to `C`."

We can think of `fold` not as a three-argument function, but as a one-argument function that:

1. Takes an argument f of type $X \rightarrow Y \rightarrow Y$ 2. Returns a function of type $\text{list } X \rightarrow Y$ that "remembers" f

When we write `fold plus`, we're giving `fold` its first argument, `plus`, and getting back a specialized function that can sum up the elements of any list of numbers. This new function still expects two more arguments: a list and a starting value.

Additional Exercises

Module `Exercises`.

Exercise: 2 stars, standard (fold_length)

Many common functions on lists can be implemented in terms of `fold`. For example, here is an alternative definition of `length`:

```
Definition fold_length {X : Type} (l : list X) : nat :=
  fold (fun _ n => S n) l 0.
```

```
Example test_fold_length1 : fold_length [4;7;0] = 3.
+
```

Prove the correctness of `fold_length`.

Hint: You may end up in a situation where you feel like `simpl` should be able to simplify `fold_length`, but does not do anything. In such cases, you can use the `unfold` tactic to inline the definition of a function prior to simplification, e.g., `unfold fold_length. simpl..` This tactic will be discussed further in the following chapter.

```
Theorem fold_length_correct : ∀ X (l : list X),
  fold_length l = length l.
Proof.
(* FILL IN HERE *) Admitted.
□
```

Exercise: 3 stars, standard (fold_map)

We can also define `map` in terms of `fold`. Finish `fold_map` below.

```
Definition fold_map {X Y: Type} (f: X → Y) (l: list X) : list Y
  (* REPLACE THIS LINE WITH ":= _your_definition_" *). Admitted.
```

Write down a theorem `fold_map_correct` stating that `fold_map` is correct, and prove it in Rocq. (Hint: again, remember that unfolding before simplifying may help.)

```
(* FILL IN HERE *)

(* Do not modify the following line: *)
Definition manual_grade_for_fold_map : option (nat×string) := None.
□
```

Exercise: 2 stars, advanced (currying)

The type $X \rightarrow Y \rightarrow Z$ can be read as describing functions that take two arguments, one of type X and another of type Y , and return an output of type Z . Recall from our discussion of partial application that this type is written $X \rightarrow (Y \rightarrow Z)$ when fully parenthesized. That is, if we have $f : X \rightarrow Y \rightarrow Z$, and we give f an input of type X , it will give us as output a function of type $Y \rightarrow Z$. If we then give that function an input of type Y , it will return an output of type Z . That is, every function in Rocq takes only one input, but some functions return a function as output. This is precisely what enables partial application, as we saw above with `plus3`.

By contrast, functions of type $X \times Y \rightarrow Z$ -- which when fully parenthesized is written $(X \times Y) \rightarrow Z$ -- require their single input to be a pair. Both arguments must be given at once; there is no possibility of partial application.

It is possible to convert a function between these two types. Converting from $X \times Y \rightarrow Z$ to $X \rightarrow Y \rightarrow Z$ is called *currying*, in honor of the logician Haskell Curry. Converting from $X \rightarrow Y \rightarrow Z$ to $X \times Y \rightarrow Z$ is called *uncurrying*.

We can define currying as follows:

```
Definition prod_curry {X Y Z : Type}
  (f : X × Y → Z) (x : X) (y : Y) : Z := f (x, y).
```

As an exercise, define its inverse, `prod_uncurry`. Then prove the theorems below to show that the two are really inverses.

```
Definition prod_uncurry {X Y Z : Type}
  (f : X → Y → Z) (p : X × Y) : Z
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.
```

As a (trivial) example of the usefulness of currying, we can use it to shorten one of the examples that we saw above:

```
Example test_map1' : map (plus 3) [2;0;2] = [5;3;5].
+
```

Thought exercise: before running the following commands, can you calculate the types of `prod_curry` and `prod_uncurry`?

```
Check @prod_curry.
Check @prod_uncurry.
```

```
Theorem uncurry_curry : ∀ (X Y Z : Type)
  (f : X → Y → Z)
  x y,
  prod_curry (prod_uncurry f) x y = f x y.
Proof.
  (* FILL IN HERE *) Admitted.
```

```
Theorem curry_uncurry : ∀ (X Y Z : Type)
  (f : (X × Y) → Z) (p : X × Y),
  prod_uncurry (prod_curry f) p = f p.
Proof.
  (* FILL IN HERE *) Admitted.
```

□

Exercise: 2 stars, advanced, optional (nth error informal)

Recall the definition of the `nth_error` function:

```
Fixpoint nth_error {X : Type} (l : list X) (n : nat) : option X :=
  match l with
  | [] => None
  | a :: l' => if n ==? 0 then Some a else nth_error l' (pred n)
  end.
```

Write a careful informal proof of the following theorem:

$\forall X \ l \ n, \text{length } l = n \rightarrow @\text{nth_error } X \ l \ n = \text{None}$

Make sure to state the induction hypothesis *explicitly*.

```
(* FILL IN HERE *)

(* Do not modify the following line: *)
Definition manual_grade_for_informal_proof : option (nat×string) :=
None.
(** *)

(** ** Church Numerals (Advanced) *)

(** The following exercises explore an alternative way of defining
    natural numbers using the _Church numerals_, which are named after
    their inventor, the mathematician Alonzo Church. We can represent
    a natural number n as a function that takes a function f as a
    parameter and returns f iterated n times. *)

Module Church. Definition cnat := forall X : Type, (X -> X) -> X -> X.

(** Let's see how to write some numbers with this notation. Iterating
    a function once should be the same as just applying it. Thus: *)

Definition one : cnat := fun (X : Type) (f : X -> X) (x : X) => f x.

(** Similarly, two should apply f twice to its argument: *)

Definition two : cnat := fun (X : Type) (f : X -> X) (x : X) => f (f
x).

(** Defining zero is somewhat trickier: how can we "apply a function
    zero times"? The answer is actually simple: just return the
    argument untouched. *)

Definition zero : cnat := fun (X : Type) (f : X -> X) (x : X) => x.

(** More generally, a number n can be written as fun X f x => f (f ...
(f x) ...), with n occurrences of f. Let's informally
    notate that as fun X f x => f^n x, with the convention that f^0 x
    is just x. Note how the doit3times function we've defined
    previously is actually just the Church representation of 3. *)

Definition three : cnat := @doit3times.

(** So n X f x represents "do it n times", where n is a Church
    numerals and "it" means applying f starting with x.

    Another way to think about the Church representation is that
    function f represents the successor operation on X, and value
    x represents the zero element of X. We could even rewrite
    with those names to make it clearer: *)

Definition zero' : cnat := fun (X : Type) (succ : X -> X) (zero : X)
=> zero. Definition one' : cnat := fun (X : Type) (succ : X -> X)
(zero : X) => succ zero. Definition two' : cnat := fun (X : Type)
(succ : X -> X) (zero : X) => succ (succ zero).

(** If we passed in S as succ and 0 as zero, we'd even get the Peano
    naturals as a result: *)
```

Example zero_church_peano : zero nat S 0 = 0. Proof. reflexivity. Qed.

Example one_church_peano : one nat S 0 = 1. Proof. reflexivity. Qed.

Example two_church_peano : two nat S 0 = 2. Proof. reflexivity. Qed.

(** One very interesting implication of the Church numerals is that we don't strictly need the natural numbers to be built-in to a functional programming language, or even to be definable with an inductive data type. It's possible to represent them purely (if not efficiently) with functions.

Of course, it's not enough just to "represent" numerals; we need to be able to do arithmetic with the representation. Show that we can by completing the definitions of the following functions. Make sure that the corresponding unit tests pass by proving them with reflexivity. *)

(** **** Exercise: 2 stars, advanced (church_scc) *)

(** Define a function that computes the successor of a Church numeral. Given a Church numeral n , its successor $\text{scc } n$ should iterate its function argument once more than n . That is, given $\text{fun } X \text{ f } x \Rightarrow f^n x$ as input, scc should produce $\text{fun } X \text{ f } x \Rightarrow f^{(n+1)} x$ as output. In other words, do it n times, then do it once more. *)

Definition scc (n : cnat) : cnat

(* REPLACE THIS LINE WITH " := _your_definition_ ." *). Admitted.

Example scc_1 : scc zero = one. Proof. (* FILL IN HERE *) Admitted.

Example scc_2 : scc one = two. Proof. (* FILL IN HERE *) Admitted.

Example scc_3 : scc two = three. Proof. (* FILL IN HERE *) Admitted.

(** *)

(** **** Exercise: 3 stars, advanced (church_plus) *)

(** Define a function that computes the addition of two Church numerals. Given $\text{fun } X \text{ f } x \Rightarrow f^n x$ and $\text{fun } X \text{ f } x \Rightarrow f^m x$ as input, plus should produce $\text{fun } X \text{ f } x \Rightarrow f^{(n+m)} x$ as output. In other words, do it n times, then do it m more times.

Hint: the "zero" argument to a Church numeral need not be just x . *)

Definition plus (n m : cnat) : cnat

(* REPLACE THIS LINE WITH " := _your_definition_ ." *). Admitted.

Example plus_1 : plus zero one = one. Proof. (* FILL IN HERE *) Admitted.

Example plus_2 : plus two three = plus three two. Proof. (* FILL IN HERE *) Admitted.

Example plus_3 : plus (plus two two) three = plus one (plus three three). Proof. (* FILL IN HERE *) Admitted.

(** *)

(** **** Exercise: 3 stars, advanced (church_mult) *)

(** Define a function that computes the multiplication of two Church numerals.

Hint: the "successor" argument to a Church numeral need not be just f .

Warning: Rocq will not let you pass `cnat` itself as the type X argument to a Church numeral; you will get a "Universe inconsistency" error. That is Rocq's way of preventing a paradox in

```

    which a type contains itself. So leave the type argument
    unchanged. *)

Definition mult (n m : cnat) : cnat
(* REPLACE THIS LINE WITH ":= _your_definition_" *). Admitted.

Example mult_1 : mult one one = one. Proof. (* FILL IN HERE *)
Admitted.

Example mult_2 : mult zero (plus three three) = zero. Proof.
(* FILL IN HERE *) Admitted.

Example mult_3 : mult two three = plus three three. Proof.
(* FILL IN HERE *) Admitted.

(** *)

(** **** Exercise: 3 stars, advanced (church_exp) *)

(** Exponentiation: *)

(** Define a function that computes the exponentiation of two Church
    numerals.

    Hint: the type argument to a Church numeral need not just be X.
    But again, you cannot pass cnat itself as the type argument.
    Finding the right type can be tricky. *)

Definition exp (n m : cnat) : cnat
(* REPLACE THIS LINE WITH ":= _your_definition_" *). Admitted.

Example exp_1 : exp two two = plus two two. Proof. (* FILL IN HERE *)
Admitted.

Example exp_2 : exp three zero = one. Proof. (* FILL IN HERE *)
Admitted.

Example exp_3 : exp three two = plus (mult two (mult two two)) one.
Proof. (* FILL IN HERE *) Admitted.

(** *)

End Church. End Exercises.

(* 2026-01-07 13:17 *)

```